NAModule2NetworkXTutorial

October 13, 2026

1 Module 2 NetworkX Tutorial

The official documentation for NetworkX can be found here.

In this tutorial, we will explore several community detection algorithms.

[1]:
```
import networkx as nx
import matplotlib.pyplot as plt
%matplotlib inline
```

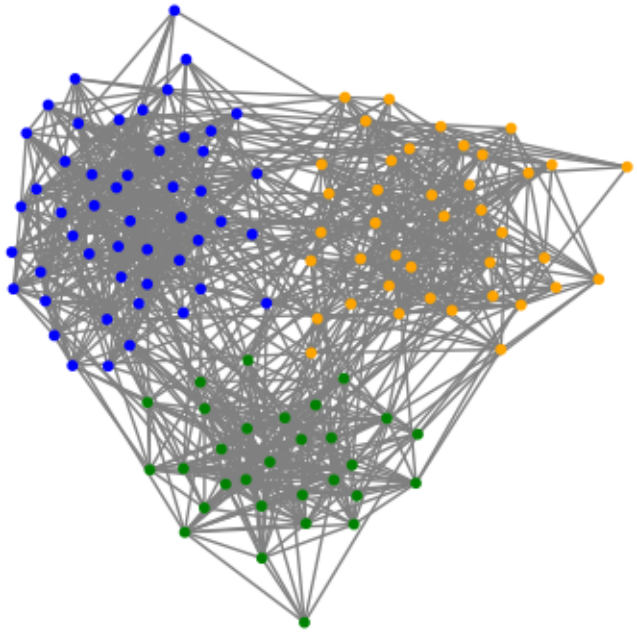
First, let's create a random graph with a three communities structure. This can be accomplished using stochastic_block_model, which we will cover in a later module. The details of how the network is generated are not important for this module.

Put simply, we generate a graph such that nodes in the same community have a higher chance of forming an edge between one another than with nodes in different communities.

[17]:
```
sizes = [50, 40, 30]
probs = [[0.25, 0.04, 0.03], [0.04, 0.3, 0.05], [0.03, 0.05, 0.40]]
G = nx.stochastic_block_model(sizes, probs, seed=0)
pos = nx.spring_layout(G)

colors = []
color_map = ['blue', 'orange', 'green']
node_idx = 0
for group_id, size in enumerate(sizes):
    colors.extend([color_map[group_id]] * size)
    node_idx += size

plt.figure(figsize=(5,5))
nx.draw_networkx(
    G, pos=pos, with_labels=False,
    node_color=colors, node_size=10,
    edge_color='grey', width=1
)
plt.axis('off')
plt.show()
```



Let's see how different community detection algorithms partition our graph.

1.1 Girvan–Newman

We can use different community detection algorithms to find a partition. For example, we can apply the Girvan–Newman method,

`networkx.algorithms.community.centrality.girvan_newman(G, most_valuable_edge=None)`

which returns a generator that produces partitions one at a time. We can retrieve a partition with the built-in function `next()` in Python.

Let's get the first partition produced by Girvan-Newman:

[5]:
```
generator = nx.algorithms.community.centrality.girvan_newman(G)
first_partition = next(generator)
len(first_partition)
```

[5]: 2

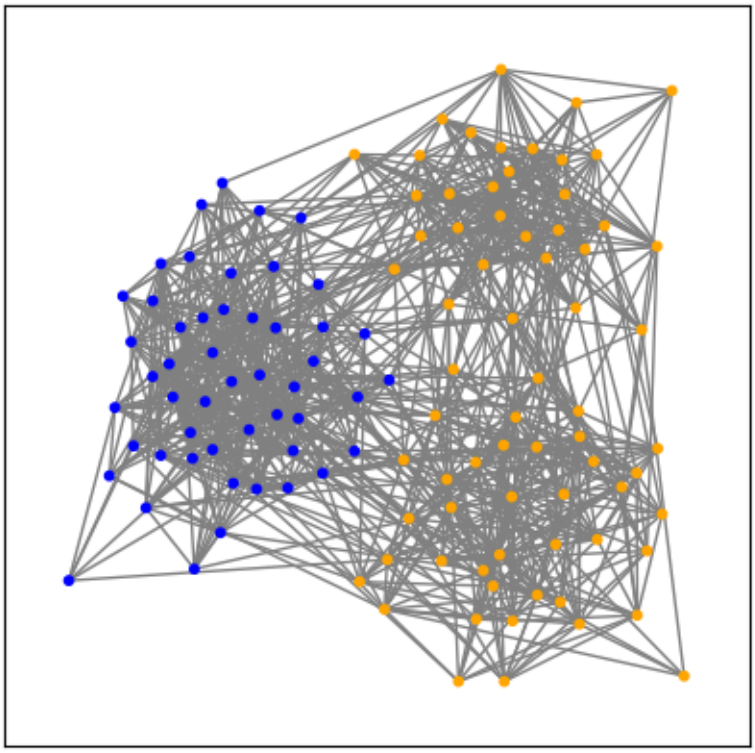
[6]:
```
print(first_partition)
```

```
({0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41,
42, 43, 44, 45, 46, 47, 48, 49}, {50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60,
```

```
61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80,
81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100,
101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116,
117, 118, 119})
```

This partition divides the network into two communities. We can color the nodes by this partition to see the outcome.

[7]:
```
color_list = ['blue', 'orange']
node_color_list = [color_list[0] if i in first_partition[0] else color_list[1]␣
 ↪for i in G.nodes]
plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='grey', width=1)
```

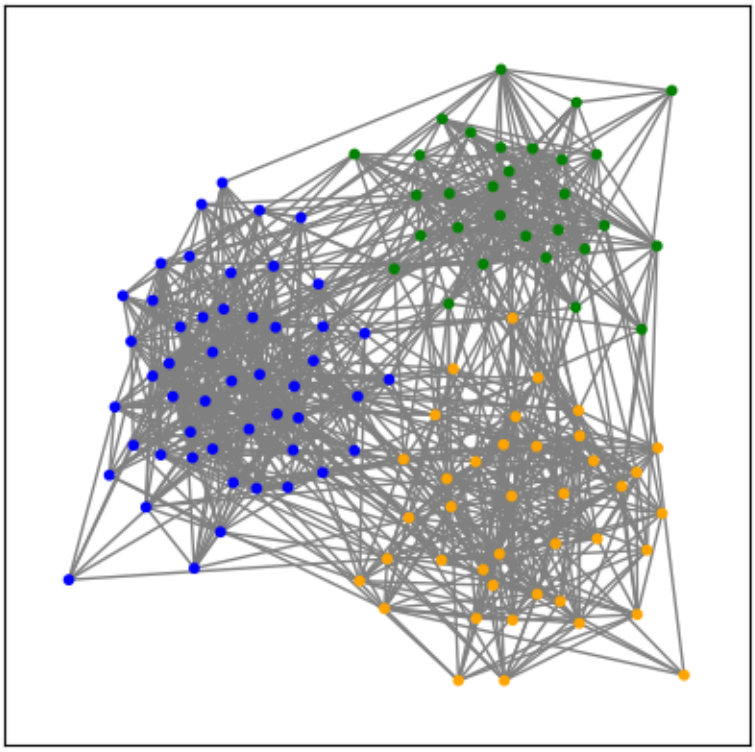


This partition has two communities, as we had configured. We can also generate the next partition to see what a third community would look like.

[8]:
```
second_partition = next(generator)
color_list = ['blue', 'orange', 'green']
for i, v in enumerate(G.nodes):
    if v in second_partition[0]:
        node_color_list[i] = color_list[0]
```

```
    elif v in second_partition[1]:
        node_color_list[i] = color_list[1]
    else:
        node_color_list[i] = color_list[2]

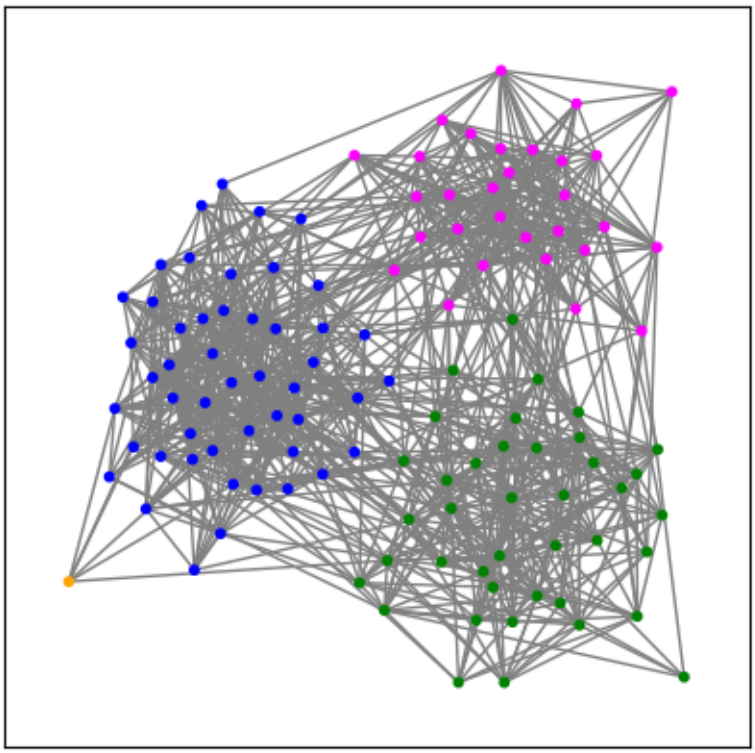
plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='grey', width=1)
```



This graph shows 3 communities, as we'd configured. Let's see what the next partition looks like

[9]:
```
third_partition = next(generator)
color_list = ['blue', 'orange', 'green', 'magenta']
for i, v in enumerate(G.nodes):
    if v in third_partition[0]:
        node_color_list[i] = color_list[0]
    elif v in third_partition[1]:
        node_color_list[i] = color_list[1]
    elif v in third_partition[2]:
        node_color_list[i] = color_list[2]
    else:
        node_color_list[i] = color_list[3]
```

```
plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='grey', width=1)
```



Clearly, partitioning the network into 4 communities rather than 3 did not generate a better partition. We will compare the modularity values at the end.

1.2 Clauset-Newman-Moore

While the Girvan–Newman algorithm begins with a single community and creates partitions by removing edges with high betweenness centrality, Agglomerative Hierarchical Clustering begins with all nodes in their own community and merges pairs of communities that lead to a higher modularity score.

The Clauset-Newman-Moore is one of such Agglomerative Hierarchical Clustering implementations.

This method returns a list of Frozensets, which are immutable Python set objects (elements in the set cannot be modified).

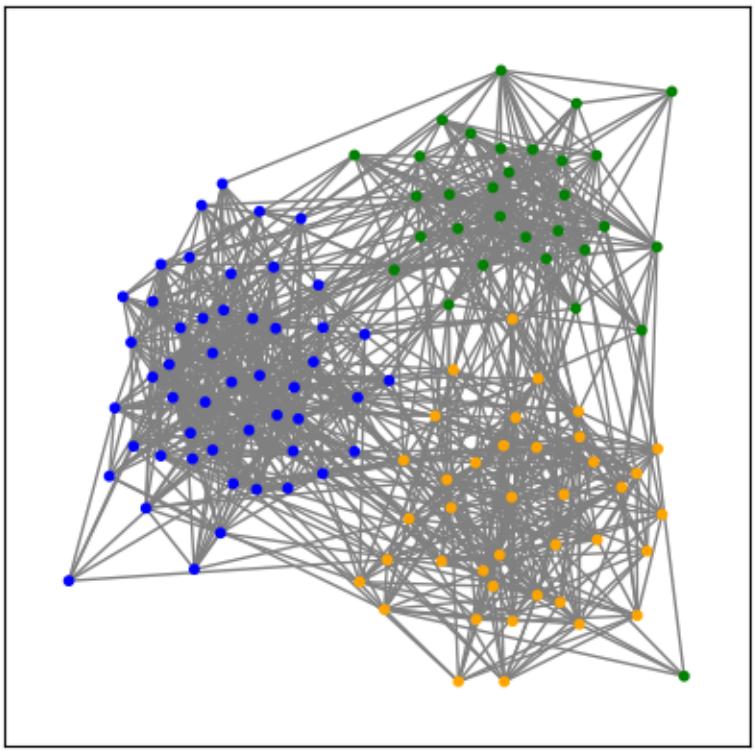
[10]:
```
gred_partition = nx.algorithms.community.greedy_modularity_communities(G)
len(gred_partition)
```

[10]: 3

As you can see, the algorithm correctly identifies the number of communities.

[11]:
```
color_list = ['blue', 'orange', 'green']
for i, v in enumerate(G.nodes):
    if v in gred_partition[0]:
        node_color_list[i] = color_list[0]
    elif v in gred_partition[1]:
        node_color_list[i] = color_list[1]
    else:
        node_color_list[i] = color_list[2]

plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='gray', width=1)
```



1.3 Label propagation

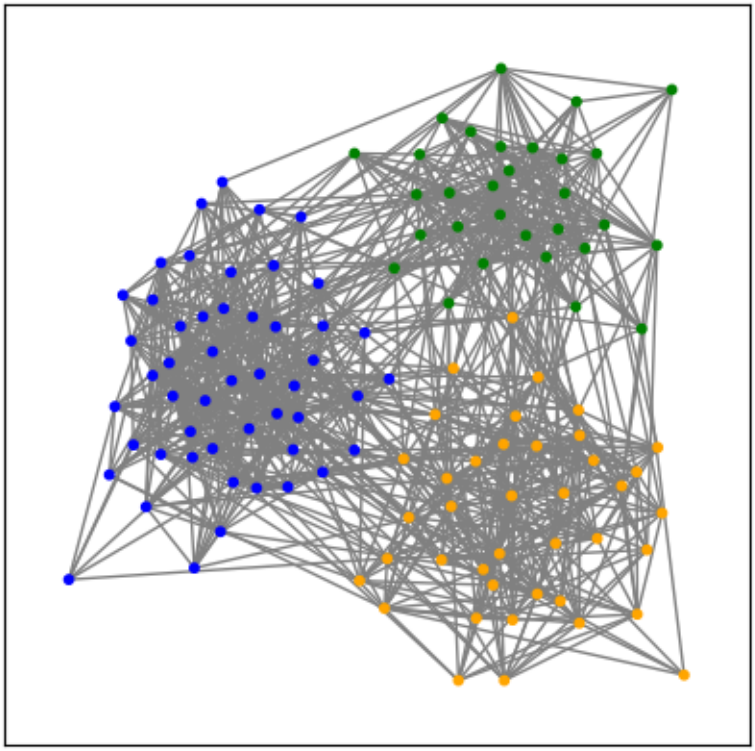
label_propagation_communities generates partitions using a label propagation algorithm. This function returns a generator as well, but each time the generator yields a set of nodes in the same group. We can convert the generator to a list of sets conveniently.

[12]:
```
lp_generator = nx.algorithms.community.label_propagation.
  ↪label_propagation_communities(G)
lp_partition = list(lp_generator)
len(lp_partition)
```

[12]:
```
3
```

[13]:
```
color_list = ['blue', 'orange', 'green']
for i, v in enumerate(G.nodes):
    if v in lp_partition[0]:
        node_color_list[i] = color_list[0]
    elif v in lp_partition[1]:
        node_color_list[i] = color_list[1]
    else:
        node_color_list[i] = color_list[2]

plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='grey', width=1)
```



1.4 Fluid communities

Note: We did not cover this in lecture.

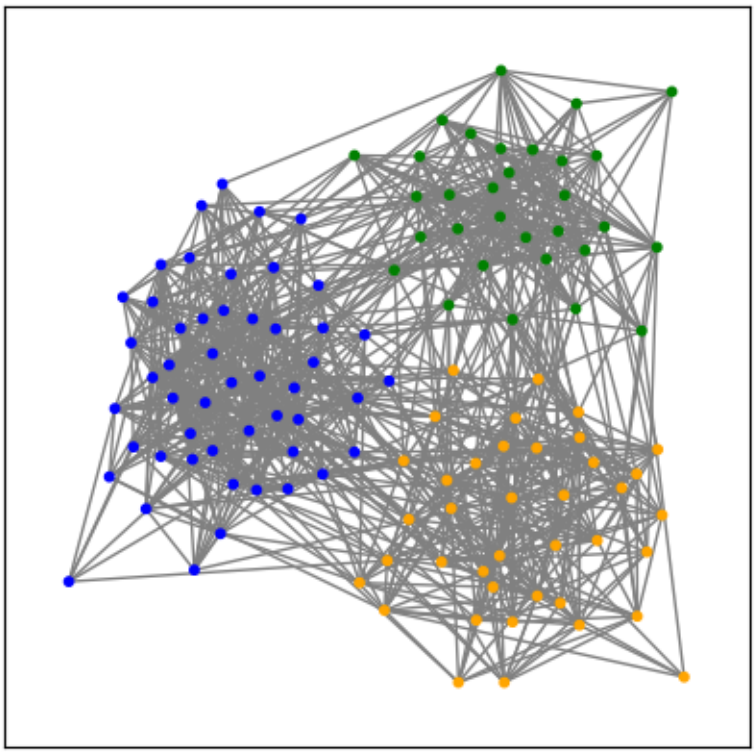
This algorithm is influenced by the interactions of fluids in an environment. Initially, communities are assigned to different nodes with density 1. During each iteration, every node is assigned to the community with the maximum density shared among its neighbors. Then the density of this community is reassigned uniformly among its members. The algorithm stops when the labels of all nodes do not change in two consecutive rounds. Note that unlike the other algorithms, you must predetermine the number of communities with input parameter .

[14]:
```
fluid_partition = list(nx.algorithms.community.asyn_fluidc(G, 3, max_iter=100,␣
↪seed=42))
len(fluid_partition)
```

[14]: 3

[15]:
```
color_list = ['blue', 'orange', 'green']
for i, v in enumerate(G.nodes):
    if v in fluid_partition[0]:
        node_color_list[i] = color_list[0]
    elif v in fluid_partition[1]:
        node_color_list[i] = color_list[1]
    else:
        node_color_list[i] = color_list[2]

plt.figure(figsize=(5,5))
nx.draw_networkx(G, pos=pos, with_labels=False,
                 node_color=node_color_list, node_size=10,
                edge_color='grey', width=1)
```



1.5 Evaluating a partition

In the previous sections, we introduced several community detection methods. However, how do we evaluate the quality of a partition?

NetworkX provides a set of metrics, which evaluate the goodness of a partition from different perspectives:

```
networkx.algorithms.community.modularity(G, communities, weight='weight')
networkx.algorithms.community.partition_quality(G, partition)
```

For example, we can compute the modularity of the partition generated by the first three iterations of the Girvan-Newman algorithm:

[16]:
```
nx.algorithms.community.modularity(G, first_partition)
```

[16]:
```
0.3489270053955369
```

[17]:
```
nx.algorithms.community.modularity(G, second_partition)
```

[17]:
```
0.4501625996381241
```

[18]:
```
nx.algorithms.community.modularity(G, third_partition)
```

[18]:
```
0.44725699708217187
```

Let's compare the coverage and the performance of the partitions generated by Girvan-Newman and Clauset-Newman-Moore:

[19]:
```
cov, perf = nx.algorithms.community.partition_quality(G, second_partition)
print(f'Girvan-Newman cov = {cov} \nGirvan-Newman per = {perf}')
```

```
Girvan-Newman cov = 0.7972027972027972
Girvan-Newman per = 0.7296918767507002
```

[20]:
```
cov, perf = nx.algorithms.community.partition_quality(G, gred_partition)
print(f'Clauset-Newman-Moore cov = {cov} \nClauset-Newman-Moore per = {perf}')
```

```
Clauset-Newman-Moore cov = 0.7937062937062938
Clauset-Newman-Moore per = 0.7301120448179271
```

Let's compare the performance of partitions generated by the label propogation and fluid algorithms:

[21]:
```
cov, perf = nx.algorithms.community.partition_quality(G, lp_partition)
print(f'label propogation cov = {cov} \nlabel propogation per = {perf}')
```

```
label propogation cov = 0.7972027972027972
label propogation per = 0.7296918767507002
```

[22]:
```
cov, perf = nx.algorithms.community.partition_quality(G, fluid_partition)
print(f'fluid_partition cov = {cov} \nfluid_partition per = {perf}')
```

```
fluid_partition cov = 0.796037296037296
fluid_partition per = 0.730672268907563
```

2 End

[1]:
```
(7 + 25 - 6) / 45
```

[1]:
```
0.5777777777777777
```

[10]:
```
(3/6 + 4/6)/2
```

[10]:
```
0.5833333333333333
```

[3]:
```
7 / (7+6)
```

[3]:
```
0.5384615384615384
```

[6]:
```
import networkx as nx

G = nx.Graph()
```

```python
# Add nodes labeled 0 to 9
G.add_nodes_from(range(10))

# Add the given edges (example)
edges = [(0, 1), (1, 2), (2, 3), (5, 6), (6, 7), (7,8), (8, 9), (4,7), (3,6),␣
 ↪(3,8), (3,9), (1,5), (1,8)]
G.add_edges_from(edges)

# Check the graph
print("Nodes:", list(G.nodes()))
print("Edges:", list(G.edges()))

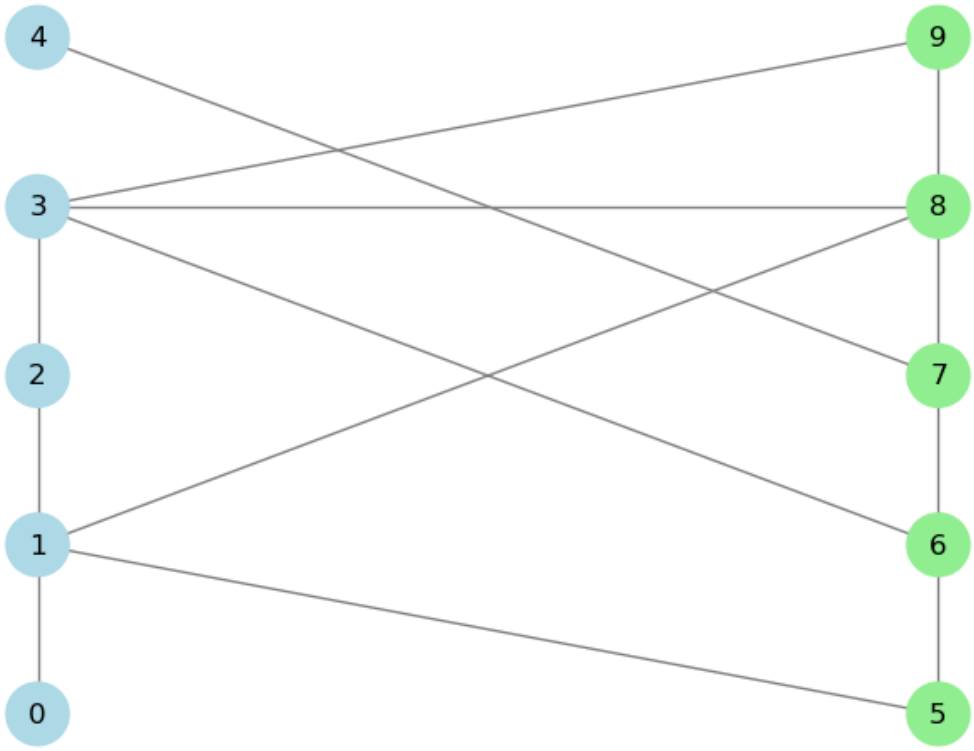
import matplotlib.pyplot as plt

left_nodes = range(5)   # Nodes 0-4
pos = nx.bipartite_layout(G, nodes=left_nodes, align="vertical")

nx.draw(
    G, pos,
    with_labels=True,
    node_color=["lightblue" if n in left_nodes else "lightgreen" for n in G.
 ↪nodes()],
    node_size=700,
    edge_color="gray"
)

plt.show()
```

```
Nodes: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
Edges: [(0, 1), (1, 2), (1, 5), (1, 8), (2, 3), (3, 6), (3, 8), (3, 9), (4, 7),
(5, 6), (6, 7), (7, 8), (8, 9)]
```



[9]:
```python
import networkx as nx
from networkx.algorithms.community import partition_quality

# Known communities
communities = [set(range(5)), set(range(5, 10))]

# Coverage and performance
coverage, performance = partition_quality(G, communities)

# Separability = internal edges / external edges
internal_edges = sum(G.subgraph(c).number_of_edges() for c in communities)
external_edges = G.number_of_edges() - internal_edges

separability = (
    internal_edges / external_edges
    if external_edges > 0 else float("inf")
)

print("Coverage:", coverage)
print("Performance:", performance)
```

```
print("Separability:", separability)
```

```
Coverage: 0.5384615384615384
Performance: 0.5777777777777777
Separability: 1.1666666666666667
```

[8]:
```
import networkx as nx
from networkx.algorithms.community import (
    partition_quality,
    louvain_communities
)

# Detect communities
communities = louvain_communities(G, seed=42)

# Convert communities to a node-to-community dictionary
partition = {
    node: i
    for i, community in enumerate(communities)
    for node in community
}

# Coverage and performance
coverage, performance = partition_quality(G, communities)

print("Communities:", communities)
print("Coverage:", coverage)
print("Performance:", performance)

internal_edges = sum(
    G.subgraph(c).number_of_edges() for c in communities
)
external_edges = G.number_of_edges() - internal_edges

separability = (
    internal_edges / external_edges if external_edges > 0 else float("inf")
)

print("Separability:", separability)
```

```
Communities: [{0, 1, 2}, {4, 5, 6, 7}, {8, 9, 3}]
Coverage: 0.6153846153846154
Performance: 0.8
Separability: 1.6
```

[ ]:
```

```